



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

**Кафедра инструментального и прикладного программного обеспечения
(ИиППО)**

КУРСОВАЯ РАБОТА

по дисциплине: Проектирование и разработка клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: «Интерактивный конструктор образовательных тестов»

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Работа представлена к защите _____ (дата) _____ /Миркин К.Л./
(подпись и ф.и.о. студента)

Руководитель: ст. преподаватель Сеницын Анатолий Васильевич

Работа допущена к защите _____ (дата) _____ /Сеницын А.В./
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____ / ст. преподаватель Сеницын А.В. /
_____ / _____ /

М. РТУ МИРЭА. 2026



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Проектирование и разработка клиент-серверных приложений
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Срок представления к защите: 18.05.2026

Руководитель: ст. преп. Сеницын Анатолий Васильевич

Тема: «Интерактивный конструктор образовательных тестов»

Исходные данные: методология двенадцати факторов, Git, Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области для выбранной темы. 2. Выбрать клиент-серверную архитектуру для разрабатываемого приложения и дать её детальное описание с помощью UML. 3. Выбрать программный стек для реализации фуллстек CRUD приложения. 4. Разработать клиентскую и серверную части приложения, реализовать авторизацию и аутентификацию пользователя, обеспечить работу с базой данных, заполнить тестовыми данными, валидировать ролевою модель на некорректные данные. 5. Провести фаззинг-тестирование. 6. Разместить исходный код клиент-серверного приложения в системе контроля версий с наличием Dockerfile и описанием структуры проекта в readme файле. 7. Развернуть клиент-серверное приложение в облаке. 8. Разработать презентацию с графическими материалами.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: Болбаков Р. Г. / Болбаков Р. Г. /, «18» февраля 2026 г.

Задание на КР выдал: Сеницын А. В. / Сеницын А. В. /, «18» февраля 2026 г.

Задание на КР получил: Миркин К. Л. / Миркин К. Л. /, «18» февраля 2026 г.

РЕФЕРАТ

Отчёт 45 с., 14 рис., 4 табл., 24 ист.

CLEAN ARCHITECTURE, FASTAPI, REST API, СЕРВЕРНАЯ ЧАСТЬ

Миркин К.Л., Курсовая работа направления подготовки «Программная инженерия» предмета «Бэкенд-разработка» на тему «Серверная часть веб-приложения для управления образовательными курсами»: М. 2026 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО)

Курсовая работа посвящена разработке серверной части веб-приложения для управления образовательными курсами. Выполнен анализ предметной области, сформированы функциональные и нефункциональные требования к системе управления обучением (LMS), проведён обзор существующих решений с построением матрицы отслеживаемости аналогов.

Проведён анализ архитектурных паттернов DDD, Clean Architecture и MVC, выполнен их сравнительный анализ и обоснован выбор Clean Architecture в качестве основного подхода к проектированию серверной части. Разработана детальная архитектура системы на основе Clean Architecture с описанием слоёв API, сервисов, репозиторий и моделей.

Реализована серверная часть на языке Python с использованием фреймворка FastAPI, СУБД PostgreSQL и библиотеки SQLAlchemy. Приведены листинги кода, описана структура REST API, слой базы данных и механизмы аутентификации. Корректность разработанной архитектуры подтверждена реализованным прототипом системы.

СОДЕРЖАНИЕ

Определения, обозначения и сокращения	7
Введение	9
1 Анализ предметной области и требований	11
1.1 Актуальность и новизна	11
1.2 Описание предметной области	11
1.3 Проблемы предметной области	12
1.4 Анализ существующих решений	13
1.4.1 Google Forms	13
1.4.2 Online Test Pad	14
1.4.3 Quizizz	15
1.4.4 Moodle (модуль Quiz)	16
1.4.5 Матрица сравнения аналогов	16
1.5 Требования к системе	18
1.5.1 Функциональные требования	18
1.5.2 Нефункциональные требования	18
1.6 Итоги анализа предметной области	19
2 Выбор архитектуры и технологического стека	21
2.1 Выбор клиент-серверной архитектуры	21
2.1.1 Архитектура клиентской части	22
2.1.2 Архитектура серверной части	23
2.1.3 ER-диаграмма	24
2.1.4 Диаграмма развёртывания	25
2.2 Выбор технологического стека	26
2.2.1 Клиентская часть	26
2.2.2 Серверная часть	27
2.2.3 Хранение данных	28
2.2.4 Инфраструктура	29
2.3 Итоги	29
3 Разработка серверной части интернет-ресурса	30
3.1 Реализация REST API на FastAPI	30
3.2 Реализация сервисного слоя	31

3.3	Слой безопасности	33
3.4	Управление конфигурацией и запуск	34
3.5	Тестирование	35
3.6	Клиентское представление	36
3.7	Итоги разработки	40
	Заключение	41
	Список использованных источников	43

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчёте применяют следующие определения, обозначения и сокращения.

API (Application Programming Interface)	– программный интерфейс, определяющий способы взаимодействия компонентов программного обеспечения.
Argon2	– алгоритм хеширования паролей, устойчивый к атакам с использованием GPU и ASIC.
Clean Architecture	– архитектурный паттерн, предложенный Робертом Мартином, обеспечивающий независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов.
DDD (Domain-Driven Design)	– подход к проектированию программного обеспечения, ориентированный на моделирование предметной области и использование единого языка (Ubiquitous Language).
Dependency Injection (DI)	– паттерн проектирования, при котором зависимости компонентов передаются извне, а не создаются внутри компонента.
DTO (Data Transfer Object)	– объект передачи данных, используемый для обмена структурированной информацией между слоями приложения.
ER-диаграмма (Entity-Relationship Diagram)	– диаграмма «сущность–связь», используемая для моделирования структуры данных и отношений между сущностями.
FastAPI	– современный веб-фреймворк для Python, ориентированный на создание REST API с поддержкой асинхронности, автоматической валидации данных и документирования.
HTTP (HyperText Transfer Protocol)	– протокол передачи данных между клиентом и сервером в веб-среде.

JWT (JSON Web Token)	– компактный и самодостаточный формат для передачи данных в виде JSON-объекта, используемый для аутентификации.
LMS (Learning Management System)	– система управления обучением, предназначенная для организации образовательного процесса, хранения материалов и контроля результатов.
MVC (Model-View-Controller)	– архитектурный паттерн, разделяющий приложение на три взаимосвязанных компонента: модель, представление и контроллер.
ORM (Object-Relational Mapping)	– технология отображения объектных структур в реляционные базы данных.
Repository	– паттерн, инкапсулирующий логику доступа к данным и предоставляющий интерфейс для операций с сущностями.
REST (Representational State Transfer)	– архитектурный стиль взаимодействия распределённых систем, использующий принципы унифицированного интерфейса и обмена представлениями ресурсов.
Service Layer	– слой бизнес-логики приложения, обрабатывающий прикладные операции и обращающийся к репозиториям.
SQL (Structured Query Language)	– язык запросов, используемый для управления реляционными базами данных.
SQLAlchemy	– библиотека ORM и инструмент работы с реляционными базами данных в Python.
СУБД	– система управления базами данных.
Фреймворк	– программный каркас, упрощающий разработку приложений и задающий архитектурные принципы.

ВВЕДЕНИЕ

Разработка серверной части современных веб-приложений представляет собой одну из ключевых дисциплин в области программной инженерии. Серверная часть (backend) отвечает за обработку бизнес-логики, хранение и предоставление данных, аутентификацию пользователей, обеспечение безопасности и масштабируемости системы. Качество архитектурных решений, принятых на этапе проектирования серверной части, напрямую определяет такие характеристики итогового продукта, как производительность, надёжность, сопровождаемость и возможность дальнейшего расширения функциональности [1].

Одной из актуальных областей применения backend-разработки являются системы управления обучением (Learning Management Systems, LMS) [2]. Такие системы обеспечивают организацию учебного процесса в цифровой среде: управление курсами и учебными материалами, проведение тестирования, отслеживание успеваемости и коммуникацию между участниками образовательного процесса. С ростом популярности дистанционного и смешанного форматов обучения возрастают требования к архитектуре подобных систем, что делает задачу проектирования и реализации их серверной части особенно значимой.

Актуальность выбранной темы определяется необходимостью разработки серверной части современной LMS-системы с применением актуальных архитектурных паттернов и технологического стека. Особое внимание уделяется Clean Architecture – подходу, обеспечивающему независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов. Применение данного паттерна позволяет создать систему, устойчивую к изменениям требований и удобную для сопровождения [3].

Цель работы – разработка серверной части веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture с использованием современных технологий и инструментов.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области и определить требования к серверной части LMS-системы;
- провести обзор и сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC, применимых к разработке серверной части;
- обосновать выбор технологического стека (FastAPI, PostgreSQL, SQLAlchemy) для реализации серверной части;
- разработать архитектуру серверной части на основе Clean Architecture с детальным описанием слоёв;
- реализовать серверную часть приложения, включая REST API, слой бизнес-логики и слой доступа к данным;
- привести листинги кода, демонстрирующие применение выбранных архитектурных решений.

Объект исследования – процессы организации и сопровождения онлайн-обучения.

Предмет исследования – архитектурные и технологические решения при разработке серверной части системы управления обучением с применением Clean Architecture.

1 Анализ предметной области и требований

1.1 Актуальность и новизна

Цифровизация образования в России приобретает устойчивый характер: по данным аналитического обзора платформ дистанционного обучения, число обучающихся в онлайн-формате выросло с 3,1 млн человек в 2019 году до 8,1 млн к 2024–2025 учебному году [4]. В этих условиях педагогам необходимо постоянно расширять цифровые компетенции и осваивать новые инструменты учебной деятельности, среди которых особое место занимают интерактивные образовательные тесты [5]. Автоматически проверяемые тесты позволяют как отслеживать текущий прогресс обучающихся (формативное оценивание), так и проводить итоговый контроль знаний, оперативно выявляя пробелы без дополнительных трудозатрат преподавателя [6].

Глобальный рынок онлайн-платформ для тестирования в 2024 году оценивается в 3,18 млрд долларов США; по прогнозам, к 2032 году он достигнет 10,14 млрд при среднегодовом темпе роста 15,6 % [7]. Высокий темп роста сегмента отражает устойчивый спрос как в академической среде, так и в корпоративном обучении.

Вместе с тем существующие инструменты создания тестов в полной мере не отвечают потребностям современного преподавателя: они либо не поддерживают математические формулы и мультимедийный контент непосредственно в теле вопроса, либо представляют собой закрытые платформы с хранением данных на серверах зарубежных компаний. Разрабатываемая система ориентирована на устранение этих ограничений: она предоставляет мультимедийный конструктор с поддержкой форматирования и математических выражений, гибким управлением доступом к тестам и возможностью развёртывания на собственной инфраструктуре.

1.2 Описание предметной области

Образовательный тест представляет собой структурированный набор заданий, предназначенных для проверки усвоения знаний обучающимися. В педагогической практике тестирование применяется как для текущего контроля успеваемости, так и для итоговой аттестации. Автором теста, как

правило, выступает преподаватель или методист; тестируемыми — студенты, школьники или иные обучающиеся.

Сложившийся процесс создания и проведения тестирования в типичной образовательной организации выглядит следующим образом:

- Преподаватель формирует тест вручную в текстовом редакторе; математические формулы оформляются как встроенные изображения или текстовые суррогаты.

- Готовый тест распространяется по электронной почте, через мессенджер или в виде ссылки на универсальную веб-форму, не предназначенную специально для тестирования.

- Медиамаериалы – рисунки, аудиозаписи, видеофрагменты – передаются отдельными файлами и не встроены в тело задания.

- Проверка ответов на вопросы со свободным вводом выполняется вручную.

- Результаты фиксируются в электронной таблице; агрегированной статистики по отдельному тесту нет.

- Тесты хранятся как разрозненные файлы: единого репозитория с поиском и повторным использованием материалов у педагога нет.

1.3 Проблемы предметной области

На основе описанного процесса выявлены следующие проблемы, обуславливающие необходимость разработки специализированного инструмента.

- 1) Отсутствует единое цифровое пространство для хранения, поиска и повторного использования образовательных тестов.

- 2) Существующие бесплатные инструменты не поддерживают нативное отображение математических формул и лёгкое форматирование непосредственно в тексте вопроса, что особенно критично для точных и естественных наук.

- 3) Медиаконтент — изображения, аудиозаписи и видеофрагменты — не интегрируется в тело вопроса, а передаётся отдельно, нарушая целостность задания.

4) Отсутствует гибкое управление доступом: нет возможности одновременно поддерживать как публичные тесты в общем каталоге, так и приватные, доступные исключительно по ссылке.

5) Вопросы со свободным вводом ответа либо не поддерживаются, либо требуют ручной проверки, что снижает ценность автоматизированного тестирования.

6) Широко распространённые платформы хранят данные пользователей и тесты на серверах зарубежных компаний, что создаёт правовые и санкционные риски для российских образовательных организаций.

7) Доступные бесплатные решения имеют устаревший интерфейс или существенно ограниченный функционал, а коммерческие платформы требуют значительных финансовых вложений.

1.4 Анализ существующих решений

Для обоснования требований к разрабатываемой системе проведён анализ четырёх широко используемых инструментов создания и проведения образовательных тестов.

1.4.1 Google Forms

Google Forms — универсальный конструктор форм от Google, применяемый в том числе для создания тестов. Поддерживает вопросы с выбором ответа, коротким и произвольным вводом. При включении режима теста автоматически проверяются ответы и выставляются баллы; результаты экспортируются в Google Sheets. Инструмент полностью бесплатен, не имеет лимита участников и интегрирован с экосистемой Google.

При этом для образовательного тестирования возможностей Google Forms недостаточно. Математические формулы в тексте вопроса не поддерживаются; изображения допускается вставить лишь в варианты ответа, но не в условие задания; аудио и видео в вопросах отсутствуют полностью. Единого каталога для публикации и поиска тестов не предусмотрено. Данные пользователей и содержимое форм хранятся на серверах Google (США), что создаёт санкционные риски для российских образовательных учреждений.

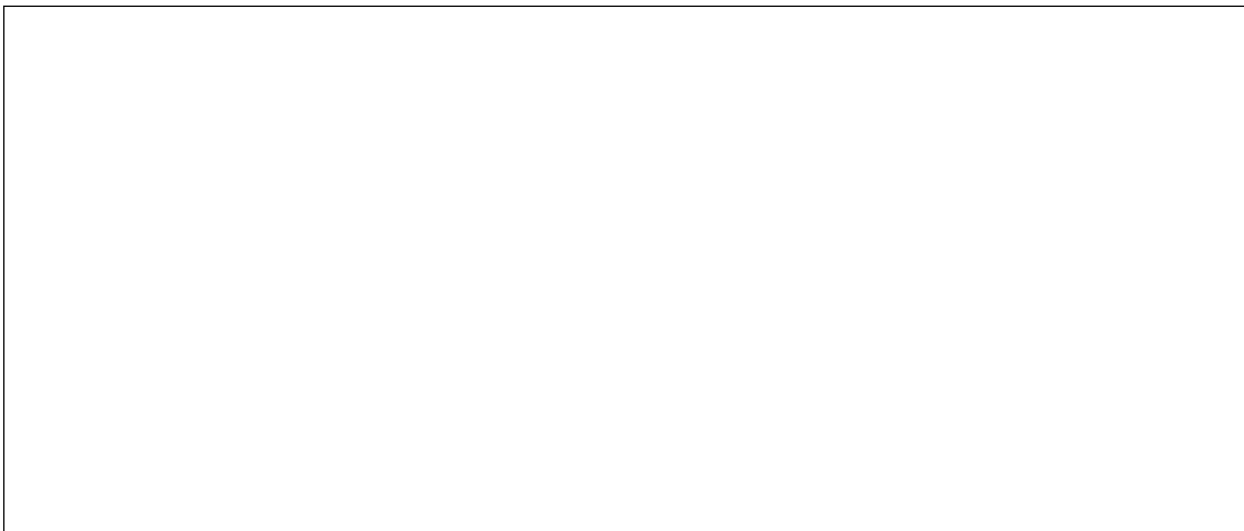


Рисунок 1 – Интерфейс Google Forms: конструктор вопросов

1.4.2 Online Test Pad

Online Test Pad — российский бесплатный онлайн-сервис для создания тестов, опросов, кроссвордов и логических игр [8]. Инструмент не требует установки; участники могут проходить тест без регистрации. Среди бесплатных отечественных решений он рекомендован педагогам как доступный инструмент для разработки интерактивных учебных материалов [5].

Вместе с тем сервис имеет устаревший интерфейс, не соответствующий современным стандартам удобства. Видео и аудио в тексте вопроса не поддерживаются; математические формулы отсутствуют. Публичного каталога с поиском по тегам нет; self-hosted развёртывание и хранение данных на собственном сервере недоступны.

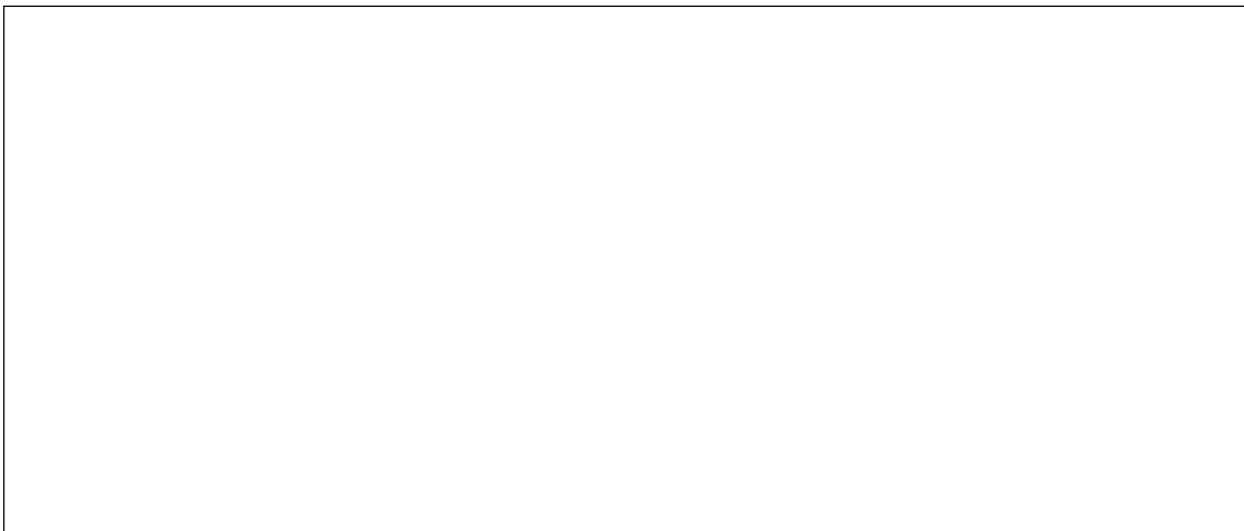


Рисунок 2 – Интерфейс Online Test Pad: конструктор теста

1.4.3 Quizizz

Quizizz — международная асинхронная платформа тестирования: каждый участник проходит задания в собственном темпе. Платформа предоставляет публичную библиотеку из более чем 30 млн материалов [9], инструменты аналитики и функции на основе искусственного интеллекта.

Наличие публичного каталога с поиском — сильная сторона платформы в сравнении с аналогами. Однако поддержка математических формул отсутствует; прикрепление аудио и видео к вопросам доступно лишь в платной подписке. Self-hosted развёртывание невозможно: данные хранятся на серверах Quizizz Inc. (США).

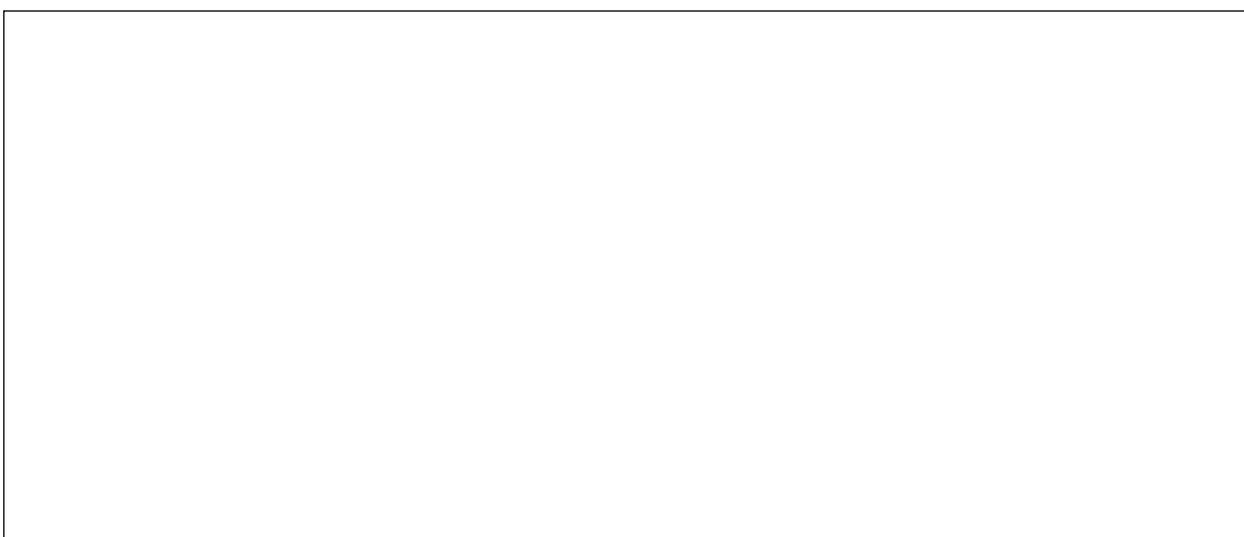


Рисунок 3 – Интерфейс Quizizz: конструктор вопросов

1.4.4 Moodle (модуль Quiz)

Moodle — открытая система управления обучением, широко применяемая в российских университетах. Встроенный модуль Quiz поддерживает около 15 типов вопросов, отображение LaTeX-формул через встроенный фильтр и настройку параметров попыток и времени. По данным библиометрического анализа публикаций в базе Scopus за 2017–2022 годы, Россия занимает первое место по числу научных работ, посвящённых Moodle, — 151 из 1045 зафиксированных документов [10].

Система обеспечивает открытый код, self-hosted развёртывание и хранение данных на собственном сервере — что является её очевидным преимуществом. Однако Moodle разработана как полноценная учебная платформа, а не специализированный конструктор тестов: создание одного теста требует прохождения большого числа экранов настройки, что создаёт высокий порог вхождения для педагогов. Единый публичный каталог тестов с тегами и поиском не предусмотрен. Встроенного хостинга для аудио и видео непосредственно в тексте вопроса нет. Развёртывание требует значительных серверных ресурсов.

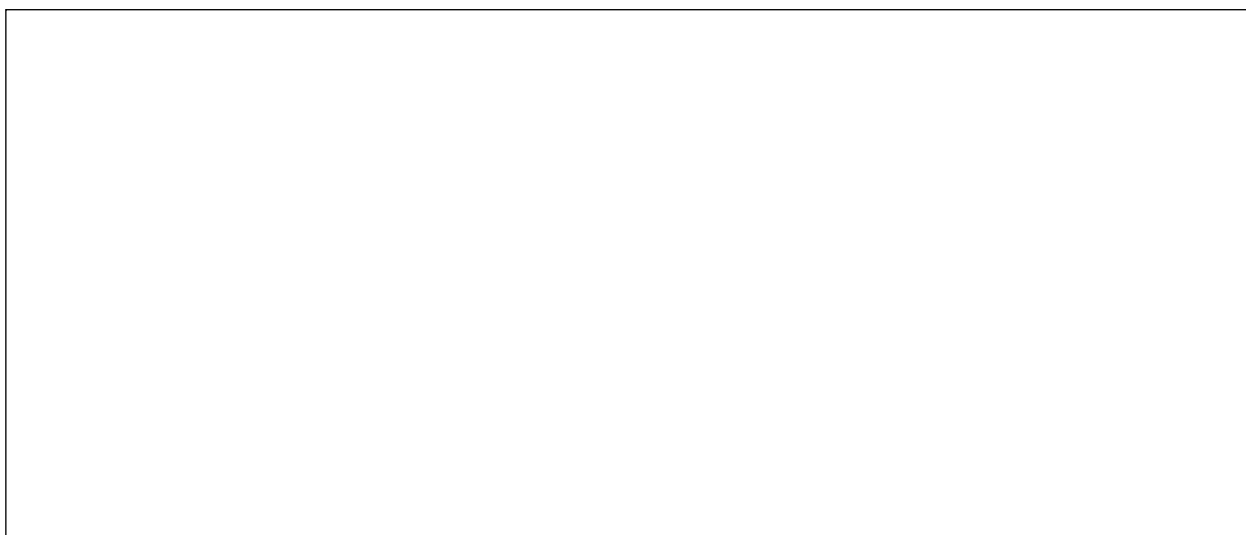


Рисунок 4 – Интерфейс Moodle: конструктор модуля Quiz

1.4.5 Матрица сравнения аналогов

В таблице 1 приведено сравнение рассмотренных инструментов по функциональным возможностям, значимым для образовательного тестирования.

Таблица 1 – Функциональное сравнение аналогов

Возможность	Google Forms	Online Test Pad	Quizizz	Moodle
Вопросы с выбором ответа	Да	Да	Да	Да
Вопросы со свободным вводом	Да	Да	Да	Да
Математические формулы	Нет	Нет	Нет	Да
Изображения в вопросе	Нет	Да	Да	Да
Аудио в вопросе	Нет	Нет	Платно	Да
Видео в вопросе	Нет	Нет	Платно	Да
Публичный каталог с тегами	Нет	Нет	Да	Нет
Приватный доступ по ссылке	Нет	Да	Да	Нет
Автоматическая проверка	Да	Да	Да	Да
Аналитика прохождений	Частично	Да	Да	Да

Таблица 2 отражает архитектурные характеристики, определяющие применимость инструментов в российских образовательных организациях.

Таблица 2 – Архитектурное сравнение аналогов

Характеристика	Google Forms	Online Test Pad	Quizizz	Moodle
Открытый исходный код	Нет	Нет	Нет	Да
Self-hosted развёртывание	Нет	Нет	Нет	Да
Данные на собственном сервере	Нет	Нет	Нет	Да
Бесплатность без ограничений	Да	Да	Частично	Да

Анализ таблиц 1 и 2 показывает, что ни один из рассмотренных инструментов не обеспечивает одновременно: поддержку математических формул, полноценный мультимедийный контент в вопросах, публичный каталог с тегами, гибкое управление доступом и self-hosted развёртывание. Moodle покрывает большинство архитектурных требований, однако уступает в простоте создания тестов и не предоставляет публичного каталога. Это определяет нишу разрабатываемой системы.

1.5 Требования к системе

1.5.1 Функциональные требования

На основании выявленных проблем сформулированы функциональные требования к системе, представленные в таблице 3.

Таблица 3 – Функциональные требования к системе

Группа	Содержание требования
Аутентификация и профиль	Регистрация нового пользователя; вход по логину и паролю; редактирование профиля.
Создание и управление тестами	Создание, редактирование и удаление теста; задание названия, описания и набора тегов; выбор режима видимости: публикация в общем каталоге или доступ по приватной ссылке.
Конструктор вопросов	Добавление, редактирование и удаление вопросов; текст с поддержкой форматирования и математических формул; тип ответа: с вариантами (один или несколько правильных) или со свободным вводом; прикрепление к вопросу медиафайлов — изображения, аудиозаписи, видеофрагмента.
Хранение медиафайлов	Загрузка и хранение медиафайлов в объектном хранилище; ограничение допустимых типов и максимального размера файла.
Поиск и каталог	Полнотекстовый поиск по названию теста; фильтрация по тегам; просмотр публичного каталога.
Прохождение теста	Последовательный показ вопросов с корректным отображением формул и медиаконтента; опциональный таймер.
Автопроверка и результаты	Автоматическая проверка ответов с вариантами; нормализованное сравнение для свободного ввода; расчёт итогового балла; постраничный разбор ответов.
Статистика автора	Количество прохождений и средний балл по созданному тесту.

1.5.2 Нефункциональные требования

Нефункциональные требования определяют эксплуатационные и инфраструктурные характеристики системы и приведены в таблице 4.

Таблица 4 – Нефункциональные требования к системе

Критерий	Метрика	Значение	Обоснование
Производительность	Время ответа (P95)	≤ 100 мс	Отклик до 100 мс воспринимается пользователем как мгновенный и обеспечивает плавную навигацию в конструкторе.
Производительность	Загрузка страницы (FCP)	≤ 3 с	Обеспечивает комфортную первичную загрузку интерфейса на типичном канале связи.
Доступность	Uptime	$\geq 99,9$ %	Бесперебойный доступ в учебные часы и в периоды контрольных мероприятий.
Безопасность	Хранение паролей	Хеширование	Пароли не хранятся в открытом виде; применяется стойкий алгоритм хеширования.
Безопасность	Доступ к приватным тестам	Только по ссылке	Тест недоступен через каталог или прямой перебор идентификаторов.
Безопасность	Загружаемые файлы	Проверка по MIME-типу и размеру	Снижает риск загрузки вредоносного контента.
Совместимость	Браузеры	Chrome, Firefox, Edge, Safari	Охват актуальных браузеров в студенческой и преподавательской среде.
Масштабируемость	Развёртывание	Контейнеризация	Контейнеризованная инфраструктура допускает горизонтальное масштабирование без изменения кода приложения.

1.6 Итоги анализа предметной области

Проведён анализ предметной области конструкторов образовательных тестов. Описан сложившийся процесс создания и проведения тестирования, на основании которого выявлены системные проблемы: отсутствие единого репозитория тестов, ограниченная поддержка мультимедиа и математических формул, трудности с управлением доступом, риски, связанные с хранением данных на серверах зарубежных компаний.

Проведён сравнительный анализ четырёх инструментов: Google Forms, Online Test Pad, Quizizz и Moodle. Ни одно из рассмотренных решений не охватывает весь необходимый набор функциональных и архитектурных характеристик. Сформированные функциональные и нефункциональные требования фиксируют целевые свойства разрабатываемой системы и служат основой для выбора технологического стека и проектирования архитектуры.

2 Выбор архитектуры и технологического стека

2.1 Выбор клиент-серверной архитектуры

Клиент-серверная архитектура обеспечивает строгое разделение ответственности между двумя ролями: клиент управляет представлением и взаимодействием с пользователем, сервер — бизнес-логикой, хранением данных и управлением доступом [11]. Для разрабатываемой системы этот подход обоснован с нескольких сторон.

Во-первых, централизованное хранение тестов, вопросов и медиафайлов на сервере позволяет любому зарегистрированному пользователю получить к ним доступ с любого устройства без локальной установки программного обеспечения. Во-вторых, автоматическая проверка ответов и управление правами на просмотр закрытых тестов выполняются исключительно на стороне сервера, что исключает клиентские манипуляции с правильными ответами. В-третьих, объектное хранилище медиафайлов (MinIO) органично встраивается в серверный контур системы.

Выбрана **трёхуровневая монолитная архитектура**: уровень представления (SPA в браузере), уровень бизнес-логики (API-сервер) и уровень данных (PostgreSQL + MinIO). Сравнение с микросервисами проводилось по критериям масштаба и состава команды. Микросервисная топология требует развитой инфраструктуры оркестрации (например, Kubernetes), отдельной системы трассировки и независимых схем развёртывания для каждого сервиса. Для проекта с одним разработчиком и компактной предметной областью (тесты, вопросы, попытки) это создаёт операционную сложность без сопоставимой выгоды. Монолит обеспечивает простоту развёртывания, прямые транзакции между таблицами и единый HTTP API, что полностью удовлетворяет требованиям первой версии продукта.

При проектировании серверной части применяются принципы методологии двенадцати факторов [12]: конфигурация хранится в переменных окружения (фактор III), зависимости объявлены явно через `requirements.txt` (фактор II), а идентичность окружений разработки и эксплуатации обеспечивается Docker Compose (фактор X).

2.1.1 Архитектура клиентской части

Клиентская часть реализуется как Single-Page Application (SPA) на React. Для организации исходного кода выбрана методология **Feature-Sliced Design** (FSD) [13]. Её ключевое преимущество — строгое правило однонаправленных зависимостей: каждый слой может импортировать сущности только из нижележащих слоёв. Это исключает циклические зависимости и делает структуру проекта предсказуемой по мере его роста.

FSD определяет семь слоёв, применённых в данном проекте:

- **app** — точка входа приложения: Router, QueryClient, глобальные провайдеры и инициализация аутентификационного состояния.
- **processes** — многошаговые пользовательские сценарии, охватывающие несколько страниц: TestPassingFlow (оркестрация прохождения теста от загрузки до отображения результата) и TestCreationFlow (пошаговый wizard создания теста с вопросами).
- **pages** — маршрутные компоненты верхнего уровня: CatalogPage, EditorPage, TakePage, ResultsPage, LoginPage. Страницы не содержат логики — они компонируют виджеты и features.
- **widgets** — переиспользуемые составные блоки интерфейса: TestCard (карточка теста в каталоге), QuestionEditor (редактор вопроса с прикреплением медиа), QuestionViewer (отображение вопроса при прохождении), ResultsPanel (разбор ответов), SearchBar (поиск + фильтрация по тегам).
- **features** — сценарии взаимодействия пользователя с системой: auth (вход/регистрация), create-test (создание и редактирование теста), take-test (прохождение теста и отправка ответов), search (поиск в публичном каталоге), typst-render (рендер форматированного текста и формул), file-upload (загрузка медиафайлов на сервер).
- **entities** — доменные объекты со своими хранилищами на Zustand: testStore, questionStore, attemptStore, userStore. Здесь определены типы данных, нормализованное состояние и простые селекторы.
- **shared** — переиспользуемая инфраструктура без доменного контекста: apiClient (настроенный axios с интерсепторами), UIKit

(компонентная база на Radix UI + Tailwind CSS), typstRenderer (WASM-обёртка над typst.ts), hooks, utils.

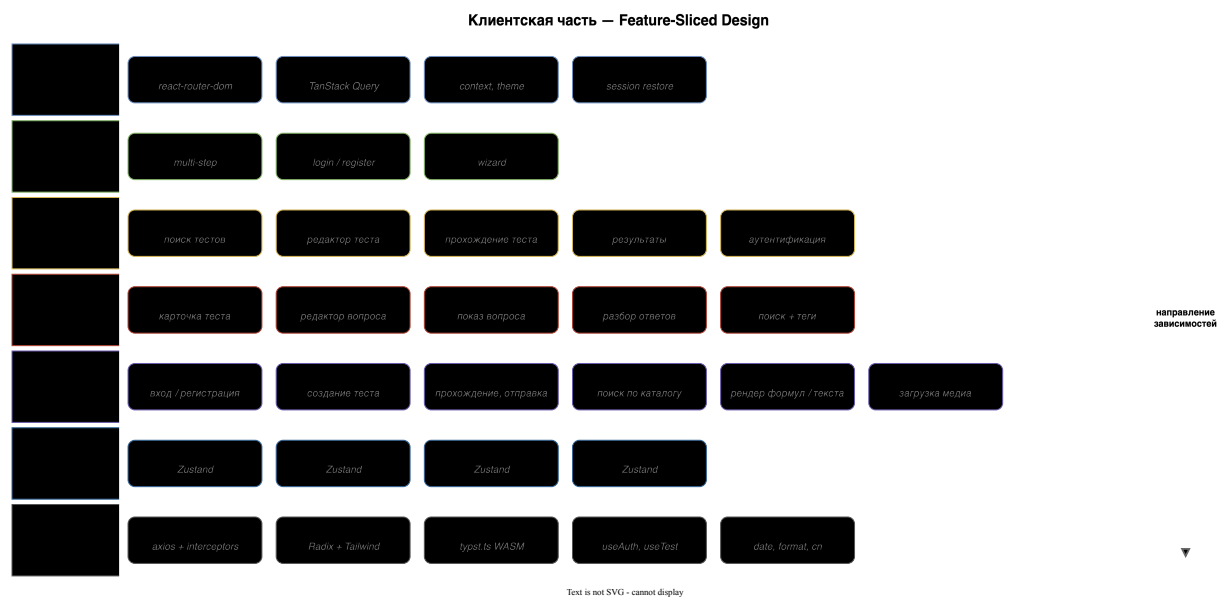


Рисунок 5 – Компонентная диаграмма клиентской части (Feature-Sliced Design)

2.1.2 Архитектура серверной части

Серверная часть строится на принципах **послойной архитектуры** с однонаправленными зависимостями между слоями [3]. Каждый слой имеет строго ограниченную зону ответственности:

- **Routers** (HTTP Layer) — приём HTTP-запросов, декларативная маршрутизация, валидация входных данных через Pydantic-схемы, формирование HTTP-ответов. Роутеры не содержат бизнес-логики — они делегируют вызовы сервисам.
- **Services** (Business Logic) — реализация прикладных сценариев: AuthService (аутентификация, хеширование паролей, выдача JWT-токенов), TestService (управление тестами и правами доступа), MediaService (валидация файлов, загрузка в S3 и генерация presigned URL), AttemptService (управление попытками прохождения и проверка ответов), CatalogService (поиск и фильтрация публичных тестов).
- **Repositories** (Data Access) — изоляция SQL-запросов от бизнес-логики через паттерн Repository. Каждый репозиторий работает с одним агрегатом и принимает асинхронную сессию SQLAlchemy.

– **Models** (ORM Entities) — декларативные классы SQLAlchemy, отображающие таблицы базы данных на Python-объекты.

Поперечные слои — **Schemas** (Pydantic DTO для входящих/исходящих данных) и **core** (конфигурация, DI-зависимости FastAPI, утилиты безопасности) — обслуживают все вышестоящие слои, не нарушая правило зависимостей.

Персистентный движок реализован через **SQLAlchemy 2.0** в режиме **asyncio**: **AsyncSession** фабрикуется через FastAPI Depends и передаётся в репозитории. Миграции схемы управляются через Alembic. Клиент MinIO (**aiobotocore**) инкапсулирован в **MediaService** и не виден репозиториям.

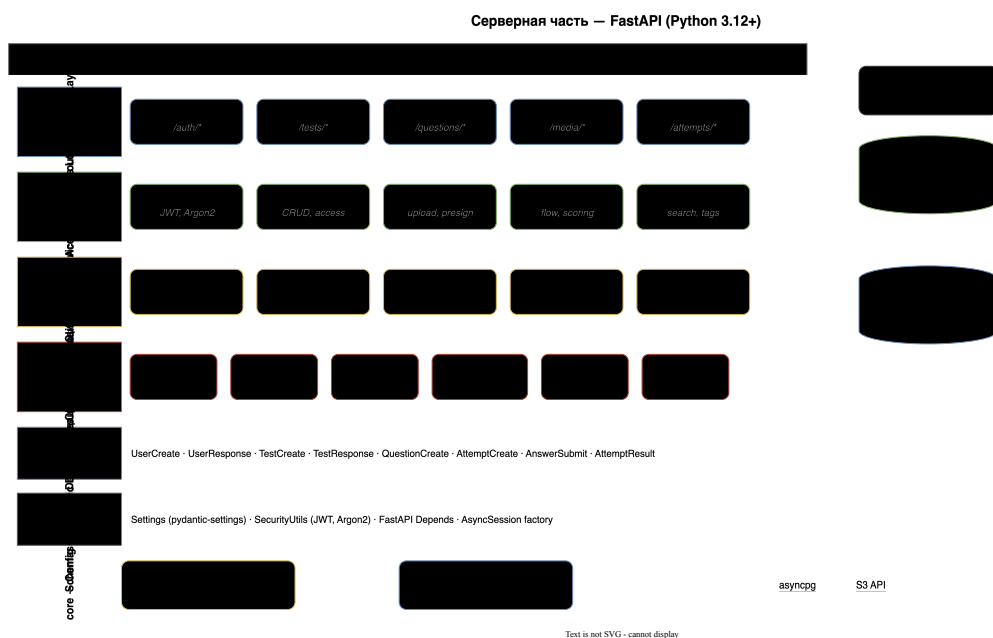


Рисунок 6 – Компонентная диаграмма серверной части (FastAPI, послойная архитектура)

2.1.3 ER-диаграмма

Схема базы данных спроектирована в соответствии с выявленными функциональными требованиями. Центральными сущностями являются **tests** и **users**: каждый тест имеет одного автора, каждый пользователь может создать неограниченное число тестов. Для тегирования применяется нормализованная модель «многие ко многим» через таблицу **test_tags**, что позволяет повторно использовать теги без дублирования.

Сущность **questions** хранит вопросы с атрибутом **type** (MCQ или свободный ввод). Для вопросов типа MCQ варианты ответа хранятся

в отдельной таблице **question_options**, что обеспечивает гибкое число вариантов. Поле `media_url` содержит ссылку на объект в MinIO — бинарный контент не хранится в реляционной базе данных.

Сущности **attempts** и **answers** фиксируют прохождение: одна попытка включает набор ответов, каждый ответ связан с конкретным вопросом. Поле `is_correct` рассчитывается сервером при приёме ответа и хранится в базе, исключая пересчёт при каждом показе результата.

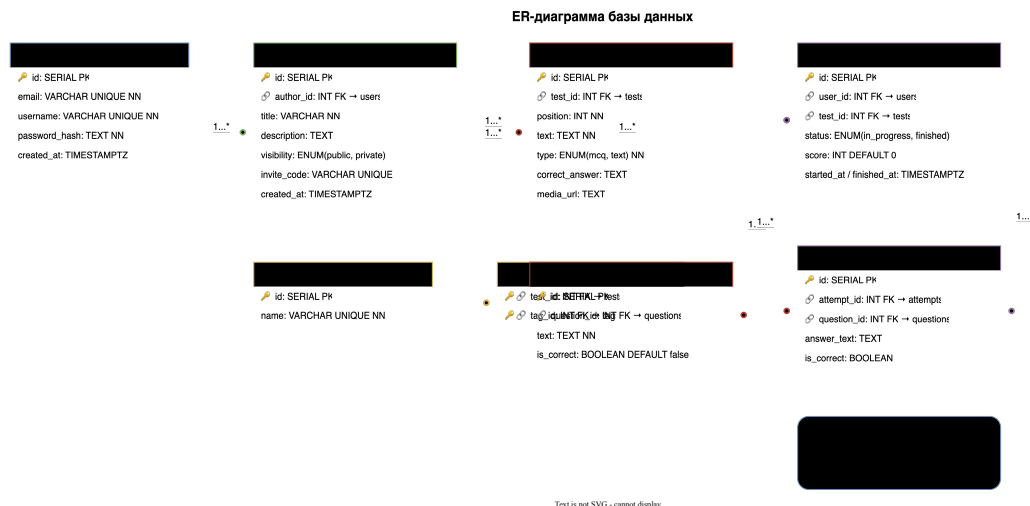


Рисунок 7 — ER-диаграмма базы данных системы

2.1.4 Диаграмма развёртывания

Система разворачивается на собственном VPS с использованием Docker Compose. Все компоненты изолированы в контейнерах и взаимодействуют через внутреннюю Docker-сеть. Nginx принимает входящий трафик на портах 80 и 443, выполняет SSL-терминирование и выступает обратным прокси-сервером: статические файлы фронтенда (скомпилированный Vite-бандл) раздаются напрямую из volume, запросы к `/api/v1/*` перенаправляются на контейнер API.

API-контейнер обращается к PostgreSQL по внутренней сети без публичного доступа к порту 5432. Медиафайлы загружаются в MinIO через S3-совместимый API (:9000); для скачивания генерируются presigned URL, по которым браузер получает файлы напрямую, минуя API.

Персистентность данных обеспечивается именованными Docker volumes: `pg_data` для PostgreSQL и `minio_data` для MinIO. Redis в MVP не

используется: для данного масштаба доступность PostgreSQL с правильными индексами обеспечивает приемлемое время отклика без промежуточного кеша.

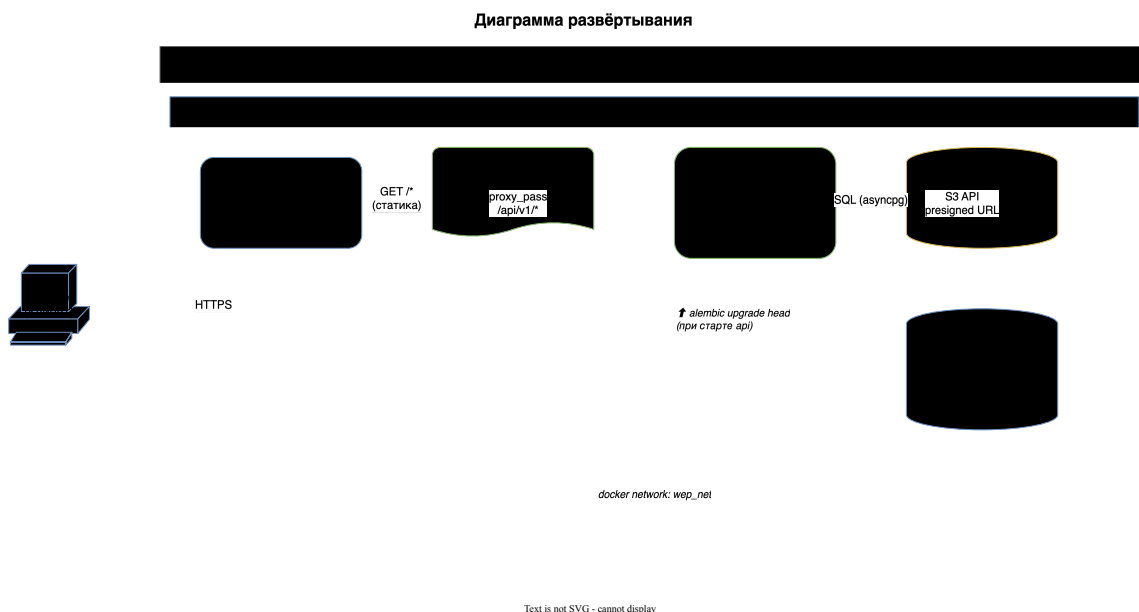


Рисунок 8 – Диаграмма развёртывания системы на VPS (Docker Compose)

2.2 Выбор технологического стека

2.2.1 Клиентская часть

Для реализации компонентного интерфейса, определённого в FSD-диаграмме, требуется декларативный UI-фреймворк с широкой экосистемой. **React 18** [14] отвечает этим требованиям: компонентная модель с хуками обеспечивает переиспользование логики, а Virtual DOM оптимизирует перерисовки при частых обновлениях (прохождение теста — динамичный сценарий с частой сменой состояния). В качестве языка выбран **TypeScript**, который устраняет целый класс ошибок на этапе компиляции, критично при сложной типизации ответов API. Инструмент сборки **Vite** обеспечивает быстрый HMR в разработке и эффективный бандл в продакшне благодаря нативным ES-модулям.

Серверное состояние (данные, полученные с API) требует кеширования, фоновой синхронизации и дедупликации запросов. **TanStack Query v4** решает эти задачи декларативно: запрос к `/api/v1/tests` кешируется на 5 минут, обновляется при фокусировке окна и переиспользуется на всех страницах без дополнительного кода. Аналоги (SWR, Apollo Client) либо менее функциональны, либо ориентированы на GraphQL. Для локального

клиентского состояния (форма создания теста, текущий вопрос) выбран **Zustand** — минималистичная библиотека без бойлерплейта, которая не требует провайдеров и не создаёт лишних ре-рендеров.

Компонентная база строится на примитивах **Radix UI** с утилитарными стилями **Tailwind CSS**. Radix предоставляет доступные (WCAG), нестилизированные компоненты (диалоги, раскрывающиеся списки, переключатели), что устраняет необходимость реализовывать поведение клавиатурной навигации вручную. Tailwind ускоряет написание стилей через именованные классы и исключает конфликты CSS-классов в монолитном приложении.

Отдельного внимания заслуживает выбор инструмента для отображения форматированного текста и математических формул в вопросах. Популярной альтернативой является **KaTeX** — быстрый рендерер LaTeX-формул для браузера. Однако KaTeX ограничен только математическими выражениями и требует от авторов знания LaTeX-синтаксиса, который сложнее для нематематиков. Вместо него выбран **typst.ts** [15] — WASM-компилятор языка Typst, запускающийся непосредственно в браузере. Typst поддерживает как математические формулы, так и произвольное форматирование текста (списки, жирный шрифт, цитаты, таблицы), имеет более простой и читаемый синтаксис, а его официальная документация даёт исчерпывающие примеры. Авторы тестов получают единый инструмент для всего содержимого вопроса, а не отдельный рендерер для математики.

2.2.2 Серверная часть

Для реализации REST API выбран **Python 3.12+** в качестве основного языка — он сочетает читаемый синтаксис, развитую асинхронную экосистему (`asyncio`) и строгую систему аннотаций типов, которая улучшает статический анализ кода через `mypy`. Среди Python-фреймворков выбран **FastAPI** [16]. Его ключевые преимущества для данного проекта: нативная `asyncio`-модель выполнения (запросы не блокируют друг друга при ожидании ответа базы данных), встроенный Dependency Injection через `Depends` (упрощает передачу сессии БД в репозитории), автоматическая генерация OpenAPI-документации из кода (снижает рассинхронизацию между кодом и спецификацией) и полная

интеграция с Pydantic для валидации. Django и Flask менее удобны для чисто API-ориентированных приложений: Django навязывает MVT-структуру и синхронный ORM, Flask требует ручной настройки большинства компонентов.

SQLAlchemy 2.0 в `asyncio`-режиме [17] является персистентным движком системы. В режиме `AsyncSession` с драйвером `asynpg` запросы к PostgreSQL не блокируют `event loop` FastAPI, что критично для производительности при конкурентных прохождении тестов. Управление миграциями через **Alembic** позволяет вносить изменения в схему базы данных под контролем версий, без потери данных.

Pydantic v2 обеспечивает декларативную валидацию входящих данных: схемы DTO описываются как обычные Python-классы с аннотациями, а FastAPI автоматически интегрирует их в маршруты. **Argon2id** выбран для хеширования паролей как алгоритм, устойчивый к GPU-атакам за счёт высокого потребления памяти. **JWT** (`python-jose`) реализует безсессионную аутентификацию: токен содержит идентификатор пользователя и срок действия, что исключает обращение к базе данных при каждом запросе.

2.2.3 Хранение данных

PostgreSQL 16 выбран в качестве реляционной СУБД [18]. Критерии выбора: строгие ACID-транзакции (корректное завершение попытки при параллельной отправке ответов), поддержка `enum`-типов и `JSONB` (для гибкого хранения метаданных при необходимости), богатый набор индексов (`GIN` для полнотекстового поиска по названиям тестов). Реляционная модель обоснована характером данных: между сущностями системы (тест, вопросы, попытки, ответы) существуют строгие ссылочные ограничения. Документоориентированные СУБД (`MongoDB`) не дают преимуществ при наличии фиксированной схемы и транзакций между несколькими коллекциями.

MinIO [19] — S3-совместимое объектное хранилище с открытым исходным кодом, разворачиваемое как Docker-контейнер. Выбор обусловлен: `self-hosted` развёртыванием (данные не покидают собственный сервер), совместимостью с Amazon S3 API (один клиент `boto3/aiobotocore` работает с любым S3-совместимым хранилищем), поддержкой `presigned URL` (клиент

загружает и скачивает файлы напрямую, минуя API) и встроенным веб-интерфейсом для администрирования. AWS S3 как альтернатива требует внешнего провайдера и порождает зависимость от зарубежной инфраструктуры.

2.2.4 Инфраструктура

Из диаграммы развёртывания следует, что система требует: воспроизводимой упаковки пяти компонентов (nginx, frontend, api, postgres, minio) и их оркестрации на одном хосте. **Docker Compose** [20] решает эту задачу декларативно: конфигурация одним файлом описывает образы, переменные окружения, сетевую изоляцию и volumes для персистентности данных. Полный разворот с нуля занимает одну команду `docker compose up -d`. Альтернативой является Kubernetes, однако для одного VPS и одного разработчика его операционная сложность (кластер, манифесты Deployment/Service/Ingress) является избыточной.

Nginx выполняет роль обратного прокси-сервера и раздаёт статику фронтенда. Статические файлы Vite-бандла хранятся в отдельном volume и монтируются в nginx, что позволяет обновлять фронтенд без перезапуска API. Конфигурация nginx покрывает SSL-терминирование, проксирование API-запросов и добавление заголовков безопасности (HSTS, X-Content-Type-Options). **Git** [20] используется для версионного контроля; репозиторий размещён на GitHub и содержит Dockerfile и описание структуры в README.

2.3 Итоги

Обоснована трёхуровневая монолитная клиент-серверная архитектура. Разработаны четыре диаграммы: компонентная клиентской части (FSD, 7 слоёв), компонентная серверной части (5 слоёв с персистентным движком), ER-диаграмма реляционной схемы (8 таблиц) и диаграмма развёртывания (Docker Compose, VPS, без Redis в MVP). Определён технологический стек: React + TypeScript + FSD + typst.ts на клиенте; Python + FastAPI + SQLAlchemy + Pydantic на сервере; PostgreSQL + MinIO для хранения данных; Docker Compose + Nginx в качестве инфраструктуры.

3 Разработка серверной части интернет-ресурса

3.1 Реализация REST API на FastAPI

Серверная часть реализована на Python (версия 3.12+) с использованием FastAPI [16,21]. Все маршруты сгруппированы по функциональным областям и зарегистрированы единым корневым роутером в `app/api/v1/__init__.py`:

- `/api/v1/auth/` – аутентификация и управление ролями
- `/api/v1/classrooms/` – классы, приглашения, чат, WebSocket
- `/api/v1/lessons/` – уроки
- `/api/v1/homework/` – домашние задания
- `/api/v1/problems/` – база задач
- `/api/v1/testing/` – приём ответов, автопроверка
- `/api/v1/statistics/` – прогресс и статистика
- `/api/v1/theory/` – теоретические материалы

При проектировании API соблюдаются стандартные принципы REST [22]. Взаимодействие строится вокруг ресурсов, доступ к которым осуществляется через стандартные HTTP-методы (GET, POST, PATCH, DELETE). Принцип взаимодействия между клиентом и сервером посредством REST API проиллюстрирован на рисунке 9.

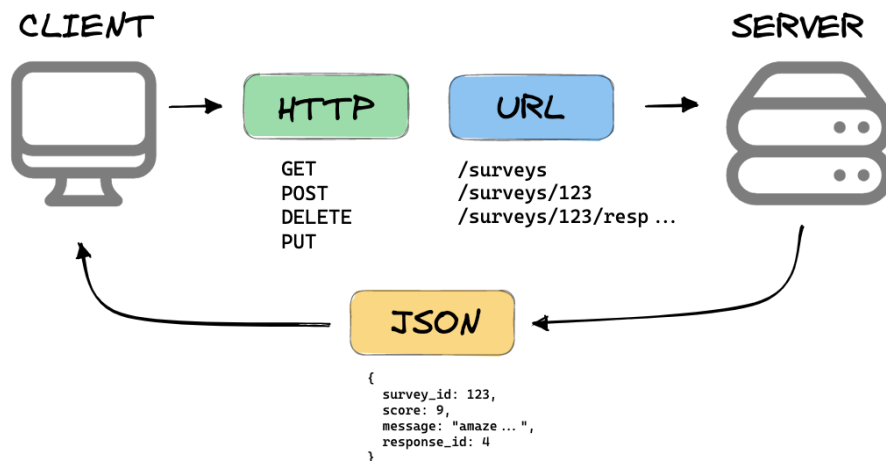


Рисунок 9 – Принцип взаимодействия через REST API

Каждый обработчик выполняет единственную задачу: валидирует входные данные через Pydantic-схему, вызывает сервис и транслирует результат в HTTP-ответ. Пример эндпоинта аутентификации приведён в листинге 1.

Листинг 1 – Реализация эндпоинта аутентификации

```
# app/api/v1/auth.py
@router.post("/login", response_model=TokenResponse)
async def login(
    credentials: LoginRequest,
    auth_service: AuthService = Depends(deps.get_auth_service),
) -> TokenResponse:
    try:
        token = await auth_service.authenticate(credentials)
    except ServiceError as exc:
        if exc.code == "inactive_user":
            raise http_errors.forbidden(exc.message, code=exc.code)
        raise
    if token is None:
        raise http_errors.unauthorized("Invalid username/email or password")
    return token
```

Листинг демонстрирует принцип явной обработки ошибок: обработчик знает семантику каждого кода и выбирает HTTP-статус самостоятельно. Незнакомое исключение не перехватывается – ответ 500 сигнализирует об ошибке на стороне сервера.

Практическая ценность такого подхода заключается в том, что поведение API становится полностью предсказуемым как для разработчика серверной части, так и для разработчика клиента. Для каждой типовой ситуации – неверные учётные данные, деактивированная учётная запись, отсутствие ресурса или нарушение ограничений доступа – обработчик формирует понятный и однотипный ответ. Это упрощает сопровождение системы и исключает скрытые преобразования ошибок, затрудняющие отладку.

3.2 Реализация сервисного слоя

Сервисный слой реализует бизнес-сценарии и не зависит от HTTP-контекста. Сервисы используют репозитории для доступа к данным, а все проверки прикладных ограничений выполняются до передачи управления на уровень ORM. Такой подход обеспечивает воспроизводимость бизнес-правил независимо от того, каким именно способом был инициирован вызов: через REST API, WebSocket или внутренний скрипт. Ключевой фрагмент сервиса аутентификации приведён в листинге 2.

Листинг 2 – Ключевой фрагмент сервиса аутентификации

```
# app/services/auth.py – метод authenticate
async def authenticate(
    self, login: LoginRequest,
) -> TokenResponse | None:
    user = await self.user_repo.get_by_username_or_email(
        login.username_or_email
    )
    if not user:
        return None
    if not user.is_active:
        raise ServiceError("User account is inactive", code="inactive_user")

    info = await self.login_repo.get_by_user_id(user.id)
    if not info or not security.verify_password(
        login.password, info.hashed_password,
    ):
        return None

    token = security.create_access_token(
        {"sub": str(user.id), "role": user.role}
    )
    return TokenResponse(
        access_token=token,
        user=UserResponse.model_validate(user),
    )
```

Сервис `TestingService` реализует ключевой сценарий автоматической проверки ответов студента. Для системы управления обучением данный сервис имеет принципиальное значение, поскольку именно он обеспечивает прикладную ценность платформы: ответ студента не просто сохраняется, а сразу сравнивается с эталоном, после чего обновляются накопленный балл, число попыток и затраченное время. Код соответствующего фрагмента приведён в листинге 3.

Листинг 3 – Фрагмент реализации автоматической проверки ответа

```
# app/services/testing.py – фрагмент метода submit_answer
async def submit_answer(
    self, answer_data: AnswerSubmit, student_user_id: int,
) -> StatisticsResponse:
    # ... [Проверка прав доступа, существования задачи и её принадлежности к
    ДЗ] ...

    stats = await self.stats_repo.get_or_create_stats(
        student.id, answer_data.homework_id, homework.max_score,
    )

    # Автоматическая проверка ответа студента
    is_correct = self._check_answer(answer_data.answer, problem.correct_answer)

    # Начисление баллов с ограничением по максимальному баллу
    new_score = (
        min(stats.score + hw_problem.points, stats.max_score) if is_correct
        else stats.score
    )

    # Фиксация попытки и затраченного времени
    updated = await self.stats_repo.update(stats.id, {
        "score": new_score, "status": "in_progress",
        "attempts_count": stats.attempts_count + 1,
        "time_spent_minutes": stats.time_spent_minutes +
answer_data.time_spent_minutes,
    })
    return StatisticsResponse.model_validate(updated)

def _check_answer(self, student: str, correct: str | None) -> bool:
    if not correct: return False
    return student.strip().lower() == correct.strip().lower()
```

3.3 Слой безопасности

Безопасность реализована на двух уровнях: хеширование паролей и аутентификация через JWT.

Пароли хешируются алгоритмом Argon2id – устойчивым к атакам с использованием GPU и ASIC, рекомендованным для хранения учётных данных [23]. JWT-токены формируются с кратким сроком действия и содержат

идентификатор пользователя и активную роль, что позволяет обнаруживать смену роли без обращения к базе данных.

Выбор Argon2id обусловлен необходимостью защищённого хранения паролей в условиях современного уровня вычислительных угроз. В отличие от простых криптографических хеш-функций, данный алгоритм учитывает не только вычислительную сложность, но и объём используемой памяти, что повышает стоимость атак перебором. Применение JWT, в свою очередь, позволяет реализовать безсессионную аутентификацию и уменьшить нагрузку на сервер при каждом запросе, сохраняя возможность централизованной проверки роли и срока действия токена.

Листинг 4 – Реализация безопасности: Argon2id и JWT

```
# app/core/security.py
_MEMORY_COST_KIB = 1024 * 64 # 64 MB

ph = argon2.PasswordHasher(
    time_cost=2, memory_cost=_MEMORY_COST_KIB,
    parallelism=1, hash_len=32, salt_len=16,
)

def verify_password(plain: str, hashed: str) -> bool:
    try:
        ph.verify(hashed, plain)
        return True
    except argon2.exceptions.VerifyMismatchError:
        return False

def create_access_token(data: dict) -> str:
    payload = {**data, "exp": utc_now() +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)}
    return jose_jwt.encode(payload, settings.SECRET_KEY,
algorithm=settings.ALGORITHM)
```

3.4 Управление конфигурацией и запуск

Конфигурация приложения хранится в переменных окружения, загружаемых через pydantic-settings [24]. Такой способ конфигурирования обеспечивает независимость параметров запуска от исходного кода и соответствует общепринятой практике построения серверных приложений. Точка входа (app/main.py) подключает middleware (CORS,

RequestIdMiddleware), глобальные обработчики ошибок и API-роутер с префиксом `/api/v1`. При наличии `REDIS_URL` в момент старта инициализируется Redis Pub/Sub-брокер для многоэкземплярного WebSocket-чата; без него приложение стартует без realtime-компонента.

Серверная часть автоматически формирует OpenAPI-спецификацию, которая используется как машинно-читаемое описание контракта API. Это важно не только для ручной проверки эндпоинтов, но и для интеграции с клиентской частью, поскольку OpenAPI-описание может выступать основой для кодогенерации, тестирования и валидации совместимости версий интерфейса. На рисунке 10 показан интерфейс OpenAPI-документации.

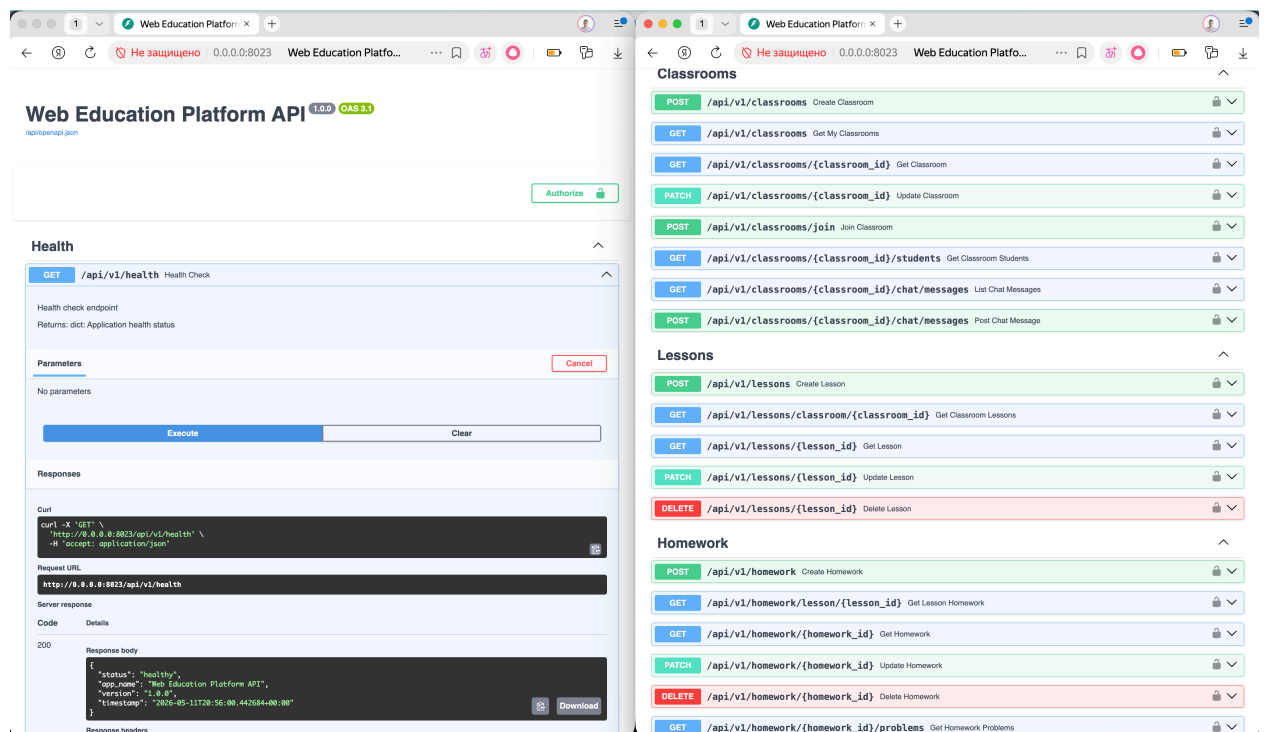


Рисунок 10 – Интерфейс автоматически сформированной OpenAPI-документации

3.5 Тестирование

Принципы Clean Architecture обеспечили практическую изоляцию сервисного слоя от веб-слоя. Тесты используют сервисы напрямую, без поднятия FastAPI. Для каждого теста создаётся отдельная схема PostgreSQL, которая накатывает таблицы из метаданных SQLAlchemy и удаляется по завершении. Это позволяет проверять бизнес-логику в реалистичных условиях при полной изоляции между тестами [17]. В составе проекта реализованы

тесты для AuthService, TestingService, ChatService, ProblemService, ResultService, TheoryService, а также для модулей realtime.

Автоматизированное тестирование выполняется с помощью pytest и служит средством проверки корректности реализованных сценариев. Тесты охватывают регистрацию и аутентификацию, автоматическую проверку ответов, подсистему обмена сообщениями, OpenAPI-контракт, расчёт прогресса и работу realtime-компонентов. На рисунке 11 показан результат выполнения набора тестов серверной части.

```
> pytest tests/ -v --tb=short --color=yes -ra

===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.4.2, pluggy-1.6.0 -- /Users/kerrodar/Dev/wep-education-platform/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/kerrodar/Dev/wep-education-platform/backend
configfile: pyproject.toml
plugins: anyio-4.12.0
collected 30 items

tests/test_auth_service.py::test_register_user_teacher[asyncio] PASSED [ 3%]
tests/test_auth_service.py::test_register_user_student[asyncio] PASSED [ 6%]
tests/test_auth_service.py::test_switch_role_creates_profile_and_updates_active_role[asyncio] PASSED [ 10%]
tests/test_auth_service.py::test_old_token_becomes_invalid_after_role_switch[asyncio] PASSED [ 13%]
tests/test_auth_service.py::test_register_duplicate_email[asyncio] PASSED [ 16%]
tests/test_auth_service.py::test_authenticate_success[asyncio] PASSED [ 20%]
tests/test_auth_service.py::test_authenticate_wrong_password[asyncio] PASSED [ 23%]
tests/test_auth_service.py::test_authenticate_nonexistent_user[asyncio] PASSED [ 26%]
tests/test_chat_pagination.py::test_message_repo_tail_and_before_id[asyncio] PASSED [ 30%]
tests/test_chat_service.py::test_chat_service_teacher_can_post_and_list[asyncio] PASSED [ 33%]
tests/test_chat_service.py::test_chat_service_rejects_non_member_student[asyncio] PASSED [ 36%]
tests/test_openapi_contract.py::test_openapi_contains_error_response_schema PASSED [ 40%]
tests/test_openapi_contract.py::test_openapi_operations_have_standard_error_responses PASSED [ 43%]
tests/test_openapi_contract.py::test_openapi_progress_endpoints_are_typed PASSED [ 46%]
tests/test_problem_service.py::test_create_problem[asyncio] PASSED [ 50%]
tests/test_problem_service.py::test_get_all_problems[asyncio] PASSED [ 53%]
tests/test_problem_service.py::test_get_problem_by_id[asyncio] PASSED [ 56%]
tests/test_problem_service.py::test_get_nonexistent_problem[asyncio] PASSED [ 60%]
tests/test_problem_service.py::test_update_problem[asyncio] PASSED [ 63%]
tests/test_problem_service.py::test_delete_problem[asyncio] PASSED [ 66%]
tests/test_realtime_auth.py::test_extract_bearer_token PASSED [ 70%]
tests/test_realtime_auth.py::test_extract_websocket_token_prefers_query_param PASSED [ 73%]
tests/test_realtime_auth.py::test_extract_websocket_token_falls_back_to_header PASSED [ 76%]
tests/test_realtime_connection_manager.py::test_connection_manager_connect_broadcast_disconnect[asyncio] PASSED [ 80%]
tests/test_realtime_presence_store.py::test_presence_store_online_filters_stale_members[asyncio] PASSED [ 83%]
tests/test_realtime_presence_store.py::test_presence_store_typing_stops_on_offline[asyncio] PASSED [ 86%]
tests/test_result_service.py::test_result_service_classroom_progress_aggregates[asyncio] PASSED [ 90%]
tests/test_result_service.py::test_result_service_student_progress_aggregates[asyncio] PASSED [ 93%]
tests/test_theory_service.py::test_theory_service_filters_unpublished_for_students[asyncio] PASSED [ 96%]
tests/test_token_role_mismatch.py::test_get_current_user_rejects_role_mismatch[asyncio] PASSED [100%]
```

Рисунок 11 – Автоматизированные тесты серверной части

3.6 Клиентское представление

Клиентская часть платформы реализована на React 18 с TypeScript [14]. Архитектура фронтенда построена по методологии Feature-Sliced Design (FSD) [13], которая структурирует код в шесть слоёв по убыванию области ответственности. Принципиальная схема данной методологии представлена на рисунке 12.

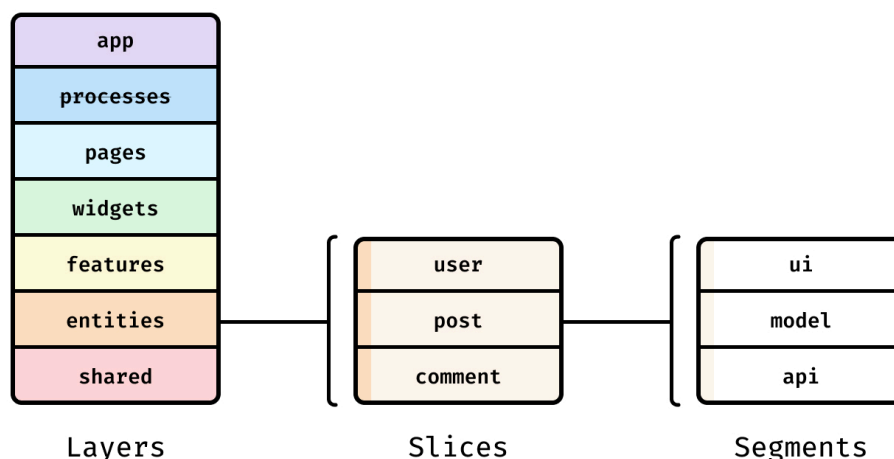


Рисунок 12 – Схема слоёв методологии Feature-Sliced Design

В рамках проекта слои распределены следующим образом:

```

src/
├─ app/      – инициализация, провайдеры, роутинг
├─ pages/    – страницы (Login, Classrooms, Lesson, ...)
├─ widgets/  – составные блоки (Header, Sidebar, ChatPanel)
├─ features/ – сценарии пользователя (JoinClassroom,
SubmitAnswer, ...)
├─ entities/ – доменные данные (Classroom, Lesson, User, Message)
└─ shared/   – переиспользуемые утилиты, UI-kit, HTTP-клиент

```

Взаимодействие с сервером ведётся через axios-клиент в `shared/api/axios.ts`. Серверный контракт описан в файле `openapi.json`, генерируемом бэкенд-командой `python -m app.scripts.export_openapi`. Управление серверным состоянием реализовано через TanStack Query v4, локальное состояние – через Zustand. Формы валидируются с помощью react-hook-form и zod. UI построен на примитивах Radix UI и стилизован через Tailwind CSS.

Требование межстраничной навигации реализуется за счёт маршрутизации в слое `app`, где определяются основные пользовательские переходы между экранами входа, списком классов, страницами уроков, домашними заданиями и представлением результатов. Такое построение навигации позволяет сохранить целостность пользовательского сценария: преподаватель переходит от списка учебных классов к конкретному уроку и связанным с ним заданиям, а студент – от структуры курса к выполнению конкретного домашнего задания и просмотру результата проверки.

Требование современного веб-дизайна обеспечивается использованием единой компонентной базы и общей системы стилизации. Применение Radix UI и Tailwind CSS позволяет поддерживать единообразие визуальных элементов, адаптивность интерфейса и предсказуемость поведения экранов, что особенно важно для образовательной платформы, ориентированной на ежедневное использование преподавателями и студентами.

Подобная организация клиентской части важна с точки зрения backend-разработки, поскольку позволяет выстроить устойчивый канал взаимодействия между сервером и пользовательским интерфейсом. Чёткое разделение страниц, сущностей и пользовательских сценариев снижает риск рассогласования между моделью данных на сервере и представлением этих данных на клиенте. Наличие единого OpenAPI-контракта дополнительно упрощает синхронизацию обеих частей системы.

Интерфейс страницы класса преподавателя должен решать две практические задачи: быстро показать состав учебной группы и дать доступ к операциям управления ею. На рисунке 13 показан интерфейс, в котором преподаватель может видеть список учеников, их успеваемость, код приглашения и элементы навигации, что позволяет быстро ориентироваться в текущем состоянии учебного процесса.

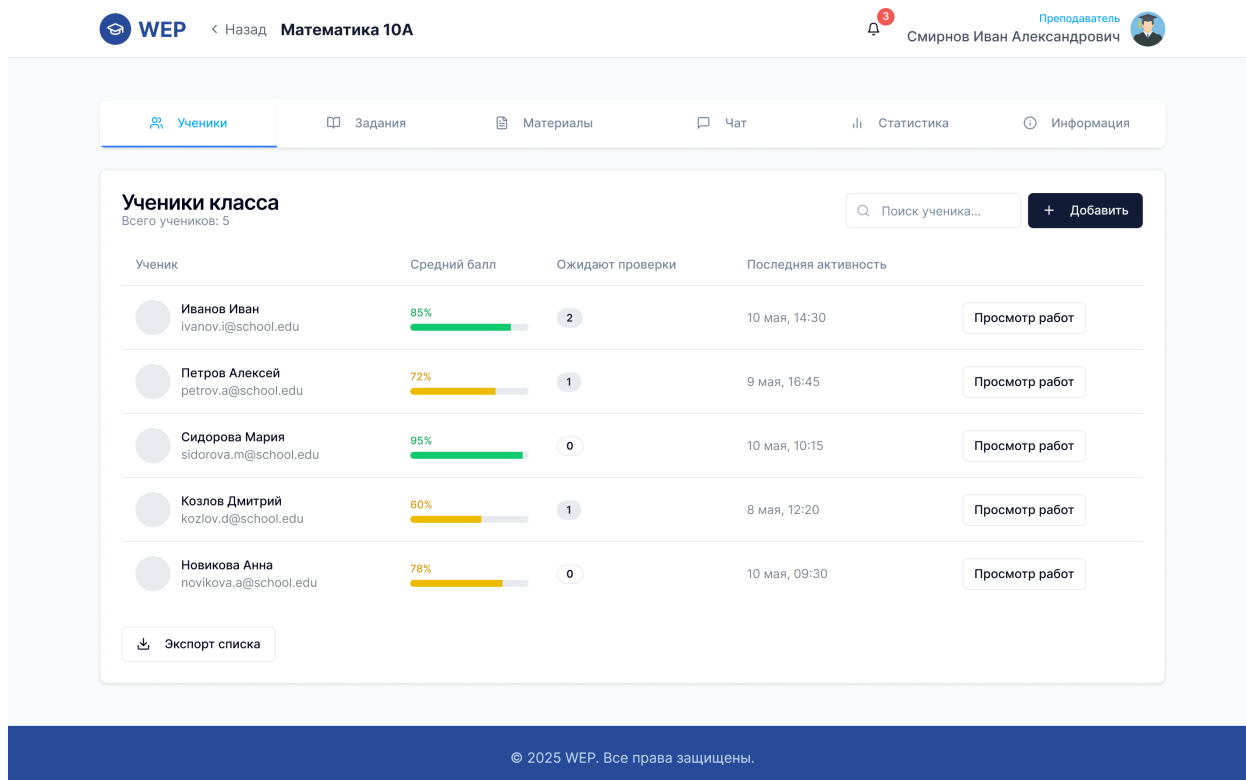


Рисунок 13 – Главная страница класса преподавателя

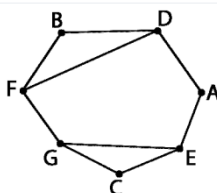
Страница выполнения домашнего задания для студента должна концентрировать навигацию по заданиям, отображать подробное условие и предоставлять удобную возможность для ввода ответа. На рисунке 14 показан интерфейс, в котором студент может перемещаться между заданиями и вводить ответы.

Открыть описание домашнего задания ▾

1 2 3 4 5

Найдите корни уравнения: $x^2 - 4 = 0$ и заполните таблицу соответствия между пунктами.

Номер пункта	Номер пункта						
	1	2	3	4	5	6	7
1		*	*	*	*		
2	*			*		*	
3	*	*					
4	*				*		*
5				*		*	*
6		*			*		
7				*	*		



Ваш ответ

Вложения:

Нет вложений

Прикрепить файл

Сохранить ответ

Рисунок 14 – Страница выполнения домашнего задания для студента

Связь фронтенда с бэкендом реализована через единый API-контракт OpenAPI: изменение серверного эндпоинта отражается в спецификации и может быть подхвачено кодогенерацией на стороне клиента, что обеспечивает согласованность интерфейсов без ручной синхронизации.

3.7 Итоги разработки

Реализована серверная часть на Python с использованием FastAPI, PostgreSQL и SQLAlchemy. REST API включает девять групп эндпоинтов, покрывающих все функциональные требования системы. Обработка ошибок построена явно: сервисы возвращают None для lookup-операций и поднимают ServiceError для командных сценариев; HTTP-статус выбирает маршрутизатор. Обеспечена безопасность: Argon2id для хранения паролей и JWT для аутентификации. Тестируемость архитектуры подтверждена набором автоматизированных тестов, работающих на уровне сервисного слоя без HTTP. Реализована клиентская часть на React + TypeScript по методологии Feature-Sliced Design, потребляющая REST API через OpenAPI-контракт.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана серверная часть веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture.

На первом этапе выполнен анализ предметной области систем управления обучением. Сформированы функциональные и нефункциональные требования к серверной части. Проведён обзор четырёх LMS-платформ – Moodle, Google Classroom, Stepik и Фоксфорд – и построена матрица отслеживаемости, выявившая ниши, в которых разрабатываемая система обеспечивает функциональные преимущества: независимая переиспользуемая база задач, встроенный realtime-чат, полноценный REST API с возможностью развёртывания на собственном сервере.

На втором этапе проведён сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC/MVT. Показано, что для разрабатываемой системы наиболее целесообразна Clean Architecture, поскольку она обеспечивает явное разделение ответственности, высокую тестируемость и независимость прикладной логики от инфраструктурных компонентов. Сформирован технологический стек: Python версии 3.12+, FastAPI, PostgreSQL, SQLAlchemy.

На третьем этапе разработана детальная архитектура серверной части. Описаны слои API, Service, Repository и Models, механизм Dependency Injection и принятый подход к обработке ошибок. Спроектирована реляционная схема базы данных, охватывающая все сущности предметной области.

На четвёртом этапе реализованы серверная и клиентская части приложения. Серверная часть включает девять групп REST-эндпоинтов, сервисный слой с автоматической проверкой ответов, слой безопасности (Argon2id, JWT) и WebSocket-чат с опциональным Redis Pub/Sub-брокером. Клиентская часть реализована на React 18 + TypeScript по методологии Feature-Sliced Design. Корректность архитектурных решений подтверждена набором автоматизированных тестов, работающих на уровне сервисного слоя без HTTP-слоя.

Полученные результаты подтверждают применимость Clean Architecture и выбранного технологического стека для построения серверной части системы управления обучением. Разработанная архитектура допускает дальнейшее расширение функциональности и масштабирование без переписывания ядра системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Минакова О.В. Программная инженерия: Основные принципы, методы и инструменты [Электронный ресурс]: учебник для вузов ISBN 978–5–507–49278–7. Санкт-Петербург: Лань, 2024. 212 с. URL: <https://e.lanbook.com/book/414989> (дата обращения: 11.05.2026).
2. Rosário A.T., Dias J.C. Learning Management Systems in Education: Research and Challenges: DOI: 10.4018/978-1-6684-4706-2.ch003 // Digital Active Methodologies for Educative Learning Management / под ред. Geada N., Jamil G.L. Hershey, PA: IGI Global, 2022. Сс. 47–77.
3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения: ISBN 978–5–4461–1903–1. 2-е изд. Санкт-Петербург: Питер, 2021. 352 с.
4. Платформы дистанционного обучения в России 2025: рейтинг, сравнение и как выбрать [Электронный ресурс]. Москва: Diasoft, 2025. URL: <https://www.diasoft.ru/about/publications/21856/> (дата обращения: 16.05.2026).
5. Анна В.Г., Екатерина И.Г. Готовность педагога к созданию и использованию интерактивных учебных материалов в профессиональной деятельности: ISSN 1994–637 // Вестник Череповецкого государственного университета. 2023. № 2. Сс. 159–169.
6. Chang C.-K., Chien L.-C.-T. Enhancing Academic Performance with Generative AI-Based Quiz Platform: DOI: 10.1109/ICALT61570.2024.00062. 2024. Сс. 193–195.
7. Online Quiz Platform Market Size, Share, Trends & Forecast [Электронный ресурс]. Verified Market Research, 2024. URL: <https://www.verifiedmarketresearch.com/product/online-quiz-platform-market/> (дата обращения: 16.05.2026).
8. Online Test Pad — онлайн-конструктор тестов, опросов, кроссвордов [Электронный ресурс]. URL: <https://onlinetestpad.com/> (дата обращения: 16.05.2026).
9. Promoting Formative Assessment with Quizizz: A Classroom Action Research Study [Электронный ресурс]. ResearchGate, 2024. URL: <https://www.researchgate.net/publication/379901904> (дата обращения: 16.05.2026).

10. Jimenez-Pitre I., Fernández J.L., Jiménez Pitre N. Usability Problems of LMS Learning Management Systems, Based on the Moodle Platform // Russian Law Journal. 2023. Т. 11, № 4.
11. Водяхо А.И. и др. Архитектурные решения информационных систем [Электронный ресурс]: учебник для вузов ISBN 978–5–507–44710–7. 3-е изд. Санкт-Петербург: Лань, 2022. 356 с. URL: <https://e.lanbook.com/book/254624> (дата обращения: 11.05.2026).
12. Wiggins A. The Twelve-Factor App [Электронный ресурс]. 2011. URL: <https://12factor.net/> (дата обращения: 16.05.2026).
13. Feature-Sliced Design: официальная документация [Электронный ресурс]. URL: <https://feature-sliced.github.io/documentation/ru/> (дата обращения: 11.05.2026).
14. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 11.05.2026).
15. typst.ts — Run Typst in JavaScript [Электронный ресурс]. URL: <https://github.com/Myriad-Dreamin/typst.ts> (дата обращения: 16.05.2026).
16. Ramírez S. FastAPI – официальная документация [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 11.05.2026).
17. SQLAlchemy 2.0 Documentation [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 11.05.2026).
18. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 11.05.2026).
19. MinIO Object Storage Documentation [Электронный ресурс]. MinIO, Inc. URL: <https://min.io/docs/> (дата обращения: 16.05.2026).
20. Docker Compose documentation [Электронный ресурс]. URL: <https://docs.docker.com/compose/> (дата обращения: 11.05.2026).
21. Елисеев А.И., Минин Ю.В. Разработка программных интерфейсов веб-приложений с использованием фреймворка FastAPI [Электронный ресурс]: учебное пособие ISBN 978–5–8265–2821–1. Тамбов: ТГТУ, 2024. 81 с. URL: <https://e.lanbook.com/book/472310> (дата обращения: 11.05.2026).
22. Алпатов А.Н. Интерфейсы прикладного программирования [Электронный ресурс]: учебное пособие ISBN 978–5–7339–2342–0. Москва: РТУ

МИРЭА, 2024. 157 с. URL: <https://e.lanbook.com/book/457043> (дата обращения: 11.05.2026).

23. Хоффман Э. Безопасность веб-приложений: ISBN 978–5–4461–1706–8. Санкт-Петербург: Питер, 2021. 336 с.
24. Pydantic V2 Documentation [Электронный ресурс]. URL: <https://docs.pydantic.dev/latest/> (дата обращения: 11.05.2026).